AI Virtual Pointer



Session (Fall 2019-Spring 2023)

Program

Bachelor of Studies in Computer Science

Submitted By

Tufail Ahmad Khan	57146-F19
Sawira Qamar	57093-F19
Muhammad Imran	3011-2108

Supervised By

Dr. Zulfiqar Ahmad

Lecturer

Department of Computer Science and IT

Hazara University, Mansehra

APPROVAL

This is to certify that we have read the project report title "**AI VIRTUAL POINTER**" submitted by the following students of BSCS Hazara University Mansehra:

S. No	Name	Roll No
1	Tufail Ahmad Khan	57146-F19
2	Muhammad Imran	3011-2108
3	Sawira Qamar	57093-F19

It is out judgment that this project report is of sufficient standards to warrant its acceptance by the Department of CS & IT, Hazara University Mansehra for the award of BSCS Degree.

COMMITTEE

Supervisor Name: _____

Supervisor Signature: _____

External Examiner _____

Head of Department CS & IT _____

ACKNOWLEDGMENT

All glory to **ALLAH**, the source of knowledge within and beyond comprehension. Who enabled us to accomplish our goals first, we express our heartiest thanks and gratefulness to almighty Allah for His divine blessings throughout our project work to complete this project successfully.

We obliged to pay our sincere and heartiest gratefulness to our supervisor **Dr. Zulfiqar Ahmad**. His timely guidance and energetic supervision, valuable advice made it possible to complete this project.

We would like to pay our sincere thanks to all our teachers for delivering latest and informative information in the courses during the degree that paved a way for us to improve our skills. We also thankful to the staff of University for their Cooperative and assisting behavior throughout.

Last but not the least we express our heartfelt gratitude to our parents for their sincere prayers, selfless support and encouragement that inspired us for all the time. Our accomplishments and success are because they believed in us, their love and support keep us motivated and confident to make this achievement possible.

Preface

Since the computer technology continues to grow up, the importance of human computer interaction is enormously increasing. Nowadays most of the mobile devices are using a touch screen technology. However, this technology is still not cheap enough to be used in desktop systems.

Creating a virtual human computer interaction device such as AI Virtual Pointer using a webcam and computer vision techniques can be an alternative way for the touch screen. Finger tracking based AI Virtual Pointer application will be designed and implemented using a regular webcam.

The motivation is to create an object tracking application to interact with the computer, and develop a virtual human computer interaction device. Many researchers in the human computer interaction and robotics fields have tried to control mouse movement, hand detection and using video devices.

TABLE OF CONTENTS

CHAPTER 1 1

INTRODUCTION 1

1.1. Introduction of the Project and its Background: 2

1.1.1. Background of the project: 2

1.2. Scope of the Project 3

1.3. Feasibility of the Project 4

1.3.1 Economic Feasibility: 4

1.3.2 Technical Feasibility: 4

1.3.3 Recurrent Cost Analysis: 5

1.4. Introduction of the Tool used in the Project 5

1.5. Thesis Organization 6

Chapter 2: REQUIREMENT SPECIFICATION 6

Chapter 3: DESIGN OF THE PROPOSED SYSTEM 6

Chapter 4: IMPLEMENTATION AND WORKING 6

Chapter 5: TESTING 6

Chapter 6: CONCLUSION 6

Chapter 7: REFERENCE MANUAL AND SCREENSHOT 6

CHAPTER 2 7

REQUIREMENT SPECIFICATION 7

2.1 Existing System 8

2.1.1. Current System 8

2.1.1. Mouse: 8

2.1.2. Graphics Tablets: 9

2.1.3. Touchscreen Devices: 9

2.1.4. Drawing Monitors: 10

2.1.5. Digital Pens: 10

2.2.1 Limitations of the existing system 11

2.2.1 Screen Size: 11

2.2.2. Lag or Latency: 11

2.2.3. Limited Customization: 11

2.2.4. Compatibility Issues: 11

2.2.5. Maintenance and Repair: 11

2.2.6. Health Issues: 11

2.2.7. Cost: 11

2.2.8. Limited Portability: 12

Chapter 3 18

Design of the proposed system 18

3.1 System Architecture 19

3.9 Scope 22

3.10 Architectural Diagram 22

Proposed system 24

3.11 Hardware 24

3.12 Software 24

3.13 User Interface 24

3.14 AI Algorithms 24

3.15 Training Data 25

3.16 Integration with Other Software 25

3.17 Hand Gesture Recognition Technology 25

3.18 Natural Hand Gestures 25

3.20 Real-Time Feedback 25

CHAPTER 4 26

TESTING 26

4.1. Black box testing 27

4.2 Module testing 27

4.2.1 Unit Testing 27

4.2.2 To accessing camera 27

4.2.3 Color detection code testing 28

4.2.4 Integration Testing 29

4.2.5 System testing 29

4.2.6 Test Cases 30

4.2.7 Automated Testing 31

4.3 Branch Testing 31

4.4.1. Use Case Access Camera 32

4.3. Use Case Movement 33

4.4. Use Case Drag 33

1.4. Tools used 34

CHAPTER 5 35

RESULTS OR SCREENSHOTS 35

5.1 Real-time optical image capture 36

5.1.2 User Interface 37

CHAPTER 6 38

Conclusion 38

6.1. Conclusions 39

6.2. Limitations 39

6.3. Future Work 40

CHAPTER 7 41

REFERENCE 41

References 42

List of Tables

Table 1: Integration Test Case 29
Table 2: Testing Video Capturing 30
Table 3: Detecting objects 30
Table 4: Joints 31
Table 5: midpoint of the line 31
Table 6: movement of the cursor 31
Table 7 Access camera 32
Table 8 Movemnet 33
Table 9 Drag 33
Table 10 Drop 34
Table 11 Tools Description 37

List of Figures

Figure 1:Mouse....8
Figure 2: Graphic Tablet....9
Figure 3: Touch Screen....9
Figure 4: Drawing Monitors....10
Figure 5: Digital Pens....10
Figure 6: Block Diagram Diagram....14
Figure 7: Use Case Diagram....15
Figure 8:Selecting Option....22
Figure 9: Architectural diagram....22
Figure 10:Flow Chart....23
Figure 11: Camera....24
Figure 12: Hand Detection....25
Figure 13:Accessing and capturing video....36
Figure 14: Circle....37
Figure 15: Eraser....37
Figure 16: Line....37
Figure 17: User Interface....37

ABBREVIATIONS

GUI	Graphical User Interface
HCI	Human-Computer Interaction
IDE	Integrated Development Environment
OpenCV	Open-Source Computer Vision
HSV	Hue, Saturation, and Values
RGB	Red, Green, and Blue

CHAPTER 1

INTRODUCTION

1.1. Introduction of the Project and its Background:

In the era of digital world, traditional art of writing is being replaced by digital art. Digital art refers to forms of expression and transmission of art form with digital form. Relying on modern science and technology is the distinctive characteristics of the digital manifestation. Traditional art refers to the art form which is created before the digital art. From the recipient to analyses, it can simply be divided into visual art, audio art, audio-visual art and audio-visual imaginary art, which includes literature, painting, sculpture, architecture, music, dance, drama and other works of art. Digital art and traditional art are interrelated and interdependent. Social development is not a people's will, but the needs of human life are the main driving force anyway. The same situation happens in art.

In the present circumstances, digital art and traditional art are inclusive of the symbiotic state, so we need to systematically understand the basic knowledge of the form between digital art and traditional art. The traditional way includes pen and paper, chalk and board method of writing. The essential aim of digital art is of building hand gesture recognition system to write digitally. Digital art includes many ways of writing like by using keyboard, touch-screen surface, digital pen, stylus, using electronic hand gloves, etc. But in this system, we are using hand gesture recognition with the use of machine learning algorithm by using python programming, which creates natural interaction between man and machine. With the advancement in technology, the need of development of natural human – computer interaction systems to replace traditional systems is increasing rapidly.

1.1.1. Background of the project:

The AI Virtual Pointer project is an exciting venture at the intersection of art and artificial intelligence. Powered by Python, this project aims to create a cutting-edge digital painting platform that revolutionizes the way artists express their creativity. By harnessing the capabilities of AI and machine learning, the platform offers a range of intuitive tools and features that enhance the painting experience.

With a user-friendly interface, artists can effortlessly navigate through the virtual canvas, selecting brushes, colors, and painting techniques to bring their visions to life. The AI algorithms integrated into the system provide intelligent assistance, generating suggestions and improving stroke quality to support the artist's creative process.

One of the standout features of the AI Virtual Pointer is the incorporation of style transfer techniques. Artists can experiment with different artistic styles, seamlessly applying the characteristics of renowned artists to their own artwork. This feature encourages

exploration and empowers artists to push their boundaries.

Real-time feedback is a crucial aspect of the platform. Artists receive immediate visual feedback, enabling them to make adjustments and decisions on the fly. The AI-generated insights offer valuable guidance and inspire new artistic directions.

The project aims to ensure compatibility and usability across various platforms, including desktop and mobile devices. This broad accessibility allows artists to engage with the platform regardless of their preferred device, fostering a diverse and vibrant community.

Python, with its rich ecosystem of AI and machine learning libraries, provides the foundation for the project's development. Libraries such as TensorFlow, PyTorch, and OpenCV offer powerful tools for implementing AI algorithms, image processing, and deep learning techniques.

Through the AI Virtual Pointer, artists can unleash their creativity and explore uncharted artistic territories. The project seeks to empower artists by combining their inherent talent with the immense capabilities of AI, opening up new avenues for self-expression.

1.2. Scope of the Project

1.2.1. User Interface

Design an intuitive and user-friendly interface for the paint app.

Include options for selecting colors, brush sizes, and different paint tools (e.g., brush, pencil, eraser, shapes).

Implement a canvas where users can draw and paint.

1.2.2. Drawing Tools

Implement basic drawing tools, such as a brush, pencil, eraser, and shapes (lines, rectangles, circles).

Allow users to adjust the brush size and select colors from a color palette.

Provide the ability to undo and redo actions for better user experience.

1.2.3. Computer Vision:

Utilize computer vision techniques to analyze and process images or video input.

Explore object detection algorithms to identify and locate objects within the input.

Implement a suitable pre-trained object detection model (e.g., YOLO, SSD, Faster R-CNN) to detect objects.

1.2.4. Virtual Painting:

Develop a feature that allows the user to select a specific object detected by the computer vision model.

Implement an algorithm to generate a virtual paint overlay on the selected object.

Provide options for customizing the virtual paint, such as color selection and brush size.
Ensure the virtual paint interacts with the object in a realistic and visually appealing manner.

1.2.5. Error Handling and User Feedback:

Handle errors and exceptions gracefully to provide a smooth user experience.
Implement informative messages or tooltips to guide users and provide feedback on their actions.

1.3. Feasibility of the Project

1.3.1 Economic Feasibility:

The economic feasibility of the AI Virtual Paint Canvas project involves assessing the financial viability and cost-effectiveness of its development and maintenance. The availability of open-source libraries and frameworks in Python significantly reduces the software development costs. Additionally, the project can leverage existing hardware infrastructure, such as personal computers, tablets, or smartphones, which are commonly owned by artists. However, there might be costs associated with acquiring or upgrading hardware if specific requirements, such as advanced GPUs for complex deep learning models, are necessary. The project can also explore potential revenue streams, such as offering premium features or collaborating with art-related businesses, to generate income and sustain its operations in the long run.

1.3.2 Technical Feasibility:

The technical feasibility of the AI Virtual Paint Canvas project involves assessing the availability of hardware and software resources required for its implementation. As a Python-based project, it can leverage the wide range of software libraries and frameworks available for AI, ML, and image processing tasks. Popular libraries like TensorFlow, PyTorch, and OpenCV provide robust tools for implementing AI algorithms, deep learning models, and image manipulation. The project can also utilize GUI frameworks such as Tkinter, PyQt, or Kivy to create an interactive user interface. The hardware requirements depend on the complexity of the AI models used. While the project can run on standard hardware, more advanced hardware configurations, including GPUs, can significantly accelerate the training and inference processes. Adequate expertise in Python programming, AI, ML, and image processing is necessary to successfully develop and maintain the project.

1.3.3 Recurrent Cost Analysis:

The recurrent cost analysis involves evaluating the ongoing expenses required for maintaining and operating the AI Virtual Paint Canvas platform. These costs include hosting and server maintenance, software updates, bug fixes, and technical support. Depending on the project's scale and user base, the costs of hosting and server infrastructure may vary. Cloud service providers like AWS, Azure, or Google Cloud Platform offer scalable options to accommodate growing user demands. Additionally, the project may require periodic updates and enhancements to improve functionality, performance, and security, which may incur development costs. Technical support for users, addressing bug reports, and ensuring a smooth user experience may also require resources. The recurrent costs should be carefully considered and balanced against the project's potential revenue or funding sources to ensure the sustainability of the platform.

Overall, the economic feasibility of the project can be favorable, especially considering the availability of open-source resources and potential revenue streams. From a technical perspective, the project can leverage the rich ecosystem of Python libraries and frameworks, with hardware requirements varying based on the project's complexity. Adequate expertise in Python programming, AI, ML, and image processing is crucial. Recurrent costs, such as hosting, maintenance, updates, and support, should be accounted for to ensure the project's long-term viability.

1.4. Introduction of the Tool used in the Project

1.4.1 Mediapipe:

This is a popular library developed by Google that provides a framework for building various perception-based applications. In this project, mediapipe is used for hand tracking and landmark detection.

1.4.2 OpenCV (cv2):

OpenCV is a widely used computer vision library that offers a range of image and video processing functions. It is employed in this project for tasks such as capturing video from the webcam, image manipulation, color conversions, and drawing on images.

1.4.3 Numpy:

NumPy is a fundamental library for scientific computing in Python. It provides powerful array operations and mathematical functions, which are utilized in this project for efficient numerical computations and manipulation of image data.

1.5. Thesis Organization

Remaining part of thesis is organized as follows:

Chapter 2: REQUIREMENT SPECIFICATION

This chapter reviews the existing literature on AI virtual mouse and related topics, including research on computer vision, machine learning, and human-computer interaction.

Chapter 3: DESIGN OF THE PROPOSED SYSTEM

This chapter describes the methods and techniques used in the project, including the development of the AI algorithm, data collection, and evaluation methods.

Chapter 4: IMPLEMENTATION AND WORKING

This chapter presents the results of the project, including a detailed analysis of the performance of the AI virtual mouse, and discusses the implications of the results for future research.

Chapter 5: TESTING

This chapter summarizes the key findings of the project and discusses potential future directions for research in AI virtual mouse technology.

Chapter 6: CONCLUSION

This chapter contains the conclusions and feature works.

Chapter 7: REFERENCE MANUAL AND SCREENSHOT

The last chapter will include Hardware pictures, screenshot and references etc

CHAPTER 2

REQUIREMENT SPECIFICATION

2.1 Existing System

In this chapter we discuss about the present system of virtual drawing. It is well-known that there are various types of physical input devices in the modern technology, the following will discuss about the types and differences about the physical devices and between virtual ways of painting on screens. A physical device can be a mouse, touch paid, track ball, physical screen pen (marker) etc. The device that helps to point and to interact with whatever that is being pointed or for painting on the screens. We are presenting the new age technology to the point and draw on the screen via moving your hand into air by using our proposed system (AI Virtual Pointer).

2.1.1. Current System

The current system has the following physical devices and techniques for painting on screens:

There are a variety of present systems for virtual drawing, each with their own strengths and limitations. Here are a few examples:

2.1.1. Mouse:

A mouse is a common input device used for digital drawing. It is simple to use and can be found on almost every computer. However, it can be difficult to create precise lines and shapes with a mouse, especially for more detailed or intricate designs.



Figure 1:Mouse

2.1.2. Graphics Tablets:

Graphics tablets are devices that allow you to draw directly onto a surface with a stylus. They offer more precision than a mouse and can often detect the pressure and angle of the stylus to create more nuanced lines and shading.



Figure 2: Graphic Tablet

2.1.3. Touchscreen Devices:



Figure 3: Touch Screen

Many tablets and smartphones come with touchscreens that allow you to draw directly onto the screen with your finger or a stylus. These devices are portable and easy to use, but they can be limited by their smaller screen size and lack of precision.

2.1.4. Drawing Monitors:

A drawing monitor is a type of graphics tablet that combines a display screen with a drawing surface. This allows you to see your drawing in real time as you create it, which can be helpful for more complex designs. Drawing monitors can be expensive, but they offer high levels of precision and accuracy.



Figure 4: Drawing Monitors

2.1.5. Digital Pens:

A Digital pen is a handheld device that allows you to create three-dimensional drawings by extruding plastic filament. They can be used to create sculptures, models, or other 3D objects. 3D pens can be difficult to use at first, but they offer a unique way to explore three-dimensional space.



Figure 5: Digital Pens

2.2.1 Limitations of the existing system

2.2.1 Screen Size:

The size of the screen or drawing surface can be a limiting factor for some systems. Small screens or surfaces can make it difficult to create detailed or complex designs, while larger screens or surfaces can be more expensive and less portable.

2.2.2. Lag or Latency:

Some systems may experience lag or latency when using a stylus or other input device. This can be especially problematic for artists who require precise or rapid movements, as lag or latency can interfere with the accuracy of their strokes.

2.2.3. Limited Customization:

Some systems may offer limited customization options for the stylus or other input device. This can make it difficult to adjust the settings to suit your individual drawing style or preferences.

2.2.4. Compatibility Issues:

Some systems may not be compatible with all types of software or operating systems. This can be a problem if you are using specialized software or have a particular workflow that requires specific hardware or software.

2.2.5. Maintenance and Repair:

Like any electronic device, systems can require maintenance or repair over time. This can be costly or time-consuming, and may require specialized knowledge or equipment to fix.

2.2.6. Health Issues:

Finally, some systems can be associated with health issues such as eye strain, neck and shoulder pain, or carpal tunnel syndrome. These issues can be mitigated through proper posture, breaks, and ergonomic equipment, but it's important to be aware of the risks associated with prolonged use of these devices.

2.2.7. Cost:

Many of the hardware devices used for virtual drawing can be expensive, especially higher-end graphics tablets or drawing monitors. This cost can be prohibitive for some artists or hobbyists who may not have the budget for these devices.

2.2.8. Limited Portability:

Some systems, such as graphics tablets or drawing monitors, are not as portable as traditional pen and paper. These devices can be bulky or require an external power source, which can make them less convenient for artists who like to work on-the-go.

2.3. Proposed System

2.3.1. Introduction:

In this proposed framework, going to utilize webcam and show unit (monitor screen). Here, will be utilizing pen or hand for drawing attractive images in front of the camera then we will attract those images, it will be shown on the presentation unit. Our mounted framework is suit for decoding time-collection pace boom alerts into extensive thing vectors. Users can make use of the pen to compose digits or make hand motions and so on may be proven at the presentation unit

2.3.2. Problem Statement:

The existing system or processes suffer from limitations and inefficiencies, necessitating the development of a new solution. The proposed system addresses these shortcomings, aiming to overcome the challenges faced in the current system.

2.3.3. Objectives:

The proposed system sets clear objectives to meet the project's requirements and goals. These objectives are designed to resolve the identified problem effectively, outlining the desired outcomes and improvements that the system aims to achieve.

2.3.4. Key Features and Functionality:

The proposed system stands out through its key features and functionalities. These distinct capabilities differentiate it from existing solutions and directly address the identified problem. The system's core functionalities have been developed to provide value and convenience to the users or stakeholders.

2.3.5. System Architecture:

The system architecture encompasses the various components, modules, or layers that constitute the proposed system. This includes the hardware, software, and network infrastructure involved, as well as any necessary external integrations. The architecture is designed to facilitate smooth interaction between different elements of the system.

2.3.6. User Experience:

A significant focus of the proposed system is on providing an exceptional user experience. It has been meticulously designed to ensure user-friendliness, intuitive navigation, and efficient workflows. The user interface (UI) and user experience (UX) principles incorporated in the system enhance usability and productivity.

2.3.7. Benefits and Impact:

The proposed system brings numerous benefits and has the potential to make a substantial impact. By streamlining processes, enhancing efficiency, and improving accuracy, it can lead to significant improvements in various aspects. The system may contribute to cost savings, better decision-making, or increased user satisfaction.

2.3.8. Implementation Plan:

The implementation plan outlines the development phases, timelines, and resource allocation for the proposed system. It addresses potential challenges and risks that may arise during the implementation process. This plan ensures a structured and organized approach to system development.

2.3.9. Evaluation and Success Criteria:

The proposed system will undergo an evaluation process to determine its success and effectiveness. This evaluation may involve gathering user feedback, conducting performance benchmarks, or employing other relevant evaluation methods. Success criteria are established to measure the system's performance against predetermined metrics.

2.4. Functional Requirements of the Proposed System

2.4.1. Drawing Tools and Canvas

The system should provide various drawing tools such as a pen, line, rectangle, circle, and eraser.

Users should be able to select and use different drawing tools.

The system should provide a canvas or drawing area where users can create and edit drawings.

2.4.2. Hand Tracking and Gesture Recognition

The system should utilize hand tracking to detect and track user hand movements.

It should recognize specific gestures to trigger actions such as selecting a tool or performing an operation.

2.4.3. Tool Selection and Customization

Users should be able to select a drawing tool from the available options.

The system should allow users to customize tool properties such as color, thickness, and opacity.

2.4.4. Drawing Operations

Users should be able to draw freehand lines, shapes (rectangles, circles), and erase markings.

The system should accurately capture and render user drawings on the canvas in real-time.

2.4.5. Erasing

The system should support erasing functionality, allowing users to erase or restore previous actions.

Users should be able to clear the canvas to start a new drawing**.**

2.4.6. User Management

The system should provide administrative features for managing user accounts, permissions, and roles.

2.4.7. Block Diagram

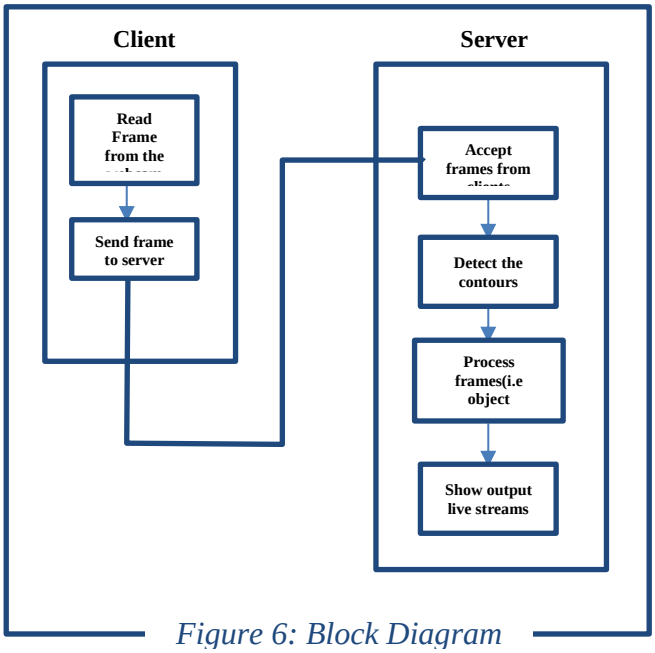


Figure 6: Block Diagram

Camera Input This is the initial source of input for project. It capture video frames from a camera connected to system. OpenCV provides functions to access the camera feed and retrieve frames.

Image Processing The camera feed may contain noise, background clutter, or lighting

variations. Image processing techniques, such as thresholding, blurring, and filtering, can help clean up the images and enhance the relevant features for hand detection.

Hand Detection Once the images are preprocessed, you'll apply hand detection algorithms to identify and track the user's hand in real-time. OpenCV offers various methods for hand detection, including background subtraction, contour analysis, and skin color segmentation. These techniques help isolate the hand region from the rest of the image.

Gesture Recognition With the hand detected, you can analyze its position, shape, and movements to recognize different hand gestures. Machine learning algorithms, such as deep learning or support vector machines (SVM), can be trained on a dataset of labeled hand gesture images to classify the gestures in real-time.

Screen Drawing After recognizing a specific gesture, it use a graphics library like Pygame to draw corresponding shapes or lines on the screen. User can track the movement of the hand and translate it into drawing commands, allowing them to create artwork or write on the screen based on their hand gestures.

Use Case Diagram

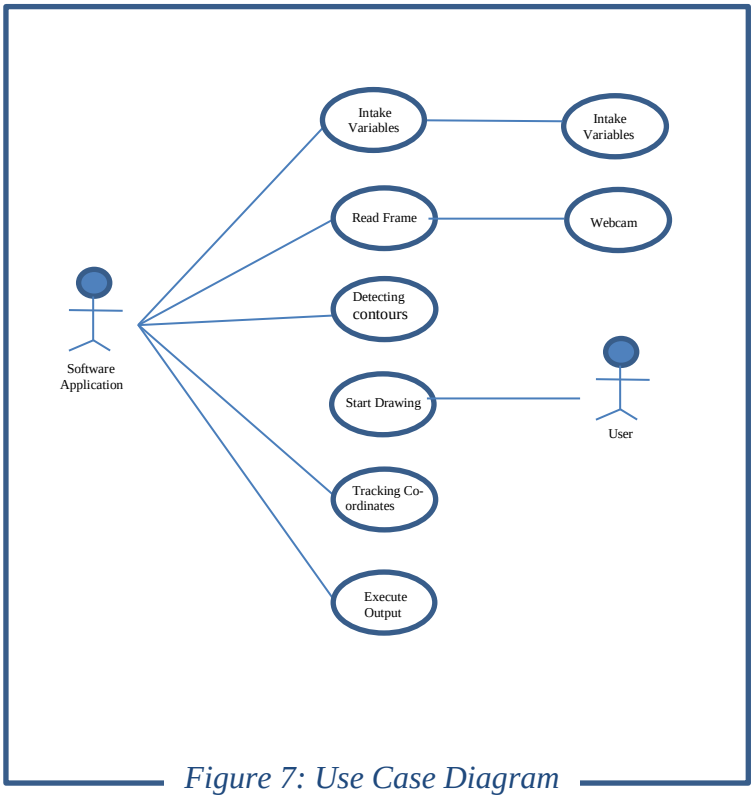


Figure 7: Use Case Diagram

2.5. Non-Functional Requirements of the Proposed System

2.5.1. Security

The system should enforce secure user authentication and access control measures to protect user data and prevent unauthorized access.

It should ensure data confidentiality, integrity, and availability through encryption and secure communication protocols.

The system should have mechanisms to detect and prevent security vulnerabilities, such as SQL injection or cross-site scripting (XSS) attacks.

2.5.2. Availability

The system should have high availability to ensure uninterrupted access and minimal downtime.

It should incorporate fault tolerance mechanisms, such as redundancy and failover systems, to minimize disruptions.

2.5.3. Performance

The system should exhibit optimal performance, responding quickly to user interactions and providing real-time rendering of drawings.

It should handle multiple users simultaneously without significant delays or performance degradation.

The system should have efficient algorithms and optimized code to minimize resource usage and maximize responsiveness.

2.5.4. Scalability

The system should be scalable to accommodate increasing user loads and growing data volumes.

It should support horizontal and vertical scalability, allowing for the addition of more users, drawing data, and system resources.

2.5.5. Usability

The system should have a user-friendly interface, with intuitive navigation and clear instructions.

It should provide helpful feedback and guidance to users, minimizing the learning curve.

The system should be accessible to users with different levels of technical expertise and accommodate users with disabilities.

2.5.6. Compatibility

The system should be compatible with different operating systems, browsers, and devices to ensure broad accessibility.

It should adhere to industry standards and support interoperability with other related software or systems.

2.5.7. Reliability

The system should be reliable and stable, minimizing the occurrence of errors or crashes.

It should have proper error handling and recovery mechanisms to ensure the system.

CHAPTER 3
DESIGN OF THE PROPOSED SYSTEM

3.1 System Architecture

The system architecture of the proposed system involves multiple components that work together to create a virtual paint application. The architecture can be described as follows

The input module captures video input from a webcam using the OpenCV library. Each frame from the webcam feed is processed to detect hand movements and gestures. The hand tracking module utilizes the Mediapipe library to track landmarks on the hand. These landmarks are essential for recognizing gestures and determining the hand's position.

The tool selection component analyzes the position of the hand to identify the selected drawing tool. By comparing the hand's coordinates with predefined regions on the screen, the code determines the current tool being used.

The drawing module performs different drawing actions based on the selected tool. It tracks the movement of the hand, particularly the index finger, to draw lines, rectangles, and circles on the canvas. Additionally, an erase tool is provided to remove drawn elements by applying a black circle.

The user interface component displays the output frame, which represents the canvas and the selected tool, in a graphical user interface (GUI) window using OpenCV. The GUI allows users to see real-time feedback of their drawing actions and the currently selected tool.

3.2 Purpose

A prototype starts with a solid design. The design is contingent on the software designer's vision and strategy for the proposed system, which in turn is based on the designer's research and comprehension of the existing system. Three distinct sections may be used to break up the design.

- Input Design
- Output Design

3.3 Algorithm Used For Hand Tracking

The MediaPipe framework handles hand gesture identification and tracking, and the OpenCV library handles computer vision. The application uses machine leaning principles to track and identify hand movements and hand tips.

3.3.1 Media Pipe

The user experience can be significantly enhanced across a range of technological domains and platforms by being able to recognize the shape and motion of hands. For instance, it can provide as the foundation for hand gesture control and sign language comprehension. It can also make it possible for digital information and material to be superimposed on top of the real world in augmented reality. Although it comes effortlessly to individuals, robust real-time hand perception is an extremely difficult computer vision problem due to the fact that hands frequently occlude themselves or each other and lack high contrast patterns. A high-fidelity hand and finger tracking solution is MediaPipe Hands. It uses machine leaning (ML) to deduce 21 3D hand landmarks formal single image. Our solution delivers real-time performance on a cell phone, and even scales to several hands, unlike existing state-of-the-art systems, which mostly rely on powerful desktop environments for inference. We anticipate that making this hand perception functionality available to a larger research and development audience will lead to the creation of innovative use cases, igniting new research directions.

3.3.2 Hand Landmark Model

After detecting the palm over the whole image, our subsequent hand landmark model uses regression, or direct coordinate prediction, to accomplish precise keypoint localization of 21 3D hand -knuckle coordinates inside the detected hand regions. The model acquires a reliable internal hand posture representation and is unaffected by self-occlusions or partially visible hands. We manually added 21 3D coordinates to around 30K real-world photos to obtain ground truth data, as shown below (we take Z-value from image depth map, if it exists per corresponding coordinate). We additionally render a high-quality synthetic hand model over a variety of backgrounds and map it to the associated 3D coordinates in order to better cover the range of possible hand poses and provide additional supervision on the nature of hand geometry.

3.3.3 OpenCV

Object detection image processing methods are included in the OpenCV computer vision library Real-time computer vision applications can be created by utilizing the OpenCV library for the Python programming language. The processing of images and videos as well as analytical techniques like face and object detection use the OpenCV library

3.3.4 NumPy

The N-dimensional array type known as ndarray is the most significant object defined in NumPy. The collection of identically categorized things is described. A zero-based index can be used to access items in the collection. ndarray's items all take up the same amount of space as a memory block. Every item in ndarray is a data-type object object (called dtype). A Python object of one of the array scalar types represents each item that is retrieved from an ndarray object (via slicing). The relationship between ndarray, data-type object (dtype), and array scalar type is depicted in the picture below.

3.4 Methodology

Based on the web camera frames that were captured, a virtual paint program was offered. The web camera sends the system the frames that it has received. Until the application is finished, the approach uses a web camera to collect each frame.

3.5 The Camera Used in the AI Virtual Pointer

The frames that have been recorded by a laptop or PC's webcam serve as the foundation for the proposed AIR CANVAS. As seen in Figure 4, the web camera will begin recording video after the video capture object is created using the Python computer vision package OpenCV. The virtual Al system receives frames from the web camera processes them.

3.6 Capturing the Video and Processing

The webcam is used by the AIR CANVAS system, and every frame is recorded up until the end of the application. As illustrated in the accompanying code, the video frames are converted from BGR to RGB in order to find the hands in the video frame by frame.

```
def findHands (self, img, draw = True)
imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
selfresults = self hands.process(imgRGB)
```

3.7 Detecting Which Finger Is Up and Performing the Particular Function

Using the tip Id of the specific finger that we located using the MediaPipe and the corresponding co-ordinates of the fingers that are up, as shown in Figure 6, we are able to determine which finger is up at this point. Then, in accordance with that determination, the specific mouse function is carried out.

3.8 Selecting an Option

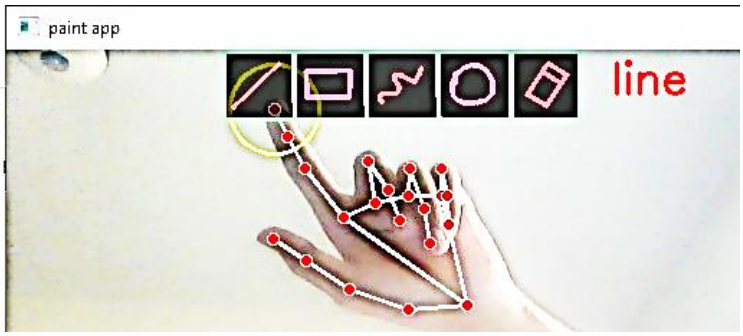


Figure 8:Selecting Option

The pynput Python module is used to program the computer to execute the right mouse button click if the middle finger with tip Id = 2 and the index finger with tip Id = 1 are both up and the distance between the two fingers is less than 40 px, as illustrated in fig.

3.9 Scope

The scope of this system is mainly used as a powerful means of communication for the deaf, which means implementing this project can help in:

1. An effective communication method that reduces mobile and laptop usage by eliminating the need to write.
2. It helps people with hearing impairments to communicate well.
3. Various purposes, such as sending messages, e-mails, etc., as the generated text can also be used for that.

3.10 Architectural Diagram



Figure 9: Architectural diagram

Flow chart of the proposed system

Start: The program is initiated, and the camera feed is initialized.

Capture Frame: A video frame is captured from the camera.

Preprocess Image: The captured frame undergoes image processing techniques to enhance

relevant features and reduce noise.

Hand Detection: Hand detection algorithms are applied to identify and track the user's hand in the preprocessed image.

Hand Detected?: Check if a hand is successfully detected in the image.

Gesture Recognition: Analyze the hand's position, shape, and movements to recognize specific hand gestures.

Gesture Type: Determine the type of gesture based on the recognized hand pattern.

Drawing Action? Check if the recognized gesture corresponds to a drawing action.

Track Hand Movement: If a drawing action is detected, track the movement of the user's hand.

Draw on Screen: Translate the hand movements into drawing commands and use a graphics library to draw shapes or lines on the screen.

Update Screen: Continuously update the screen with the drawn shapes as the user continues to move their hand.

Repeat: Return to capturing a new frame and repeat the process to provide real-time interaction and drawing capability.

End: The program ends when the user decides to exit.

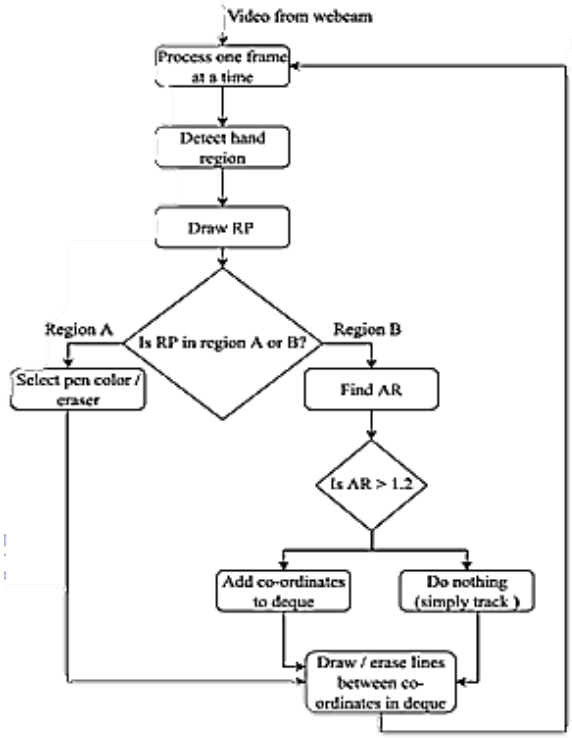


Figure 10:Flow Chart

Proposed system

3.11 Hardware

The system would require a computer with a webcam and a stylus or other input device. The computer should have sufficient processing power to run the necessary software, as well as a high-quality display for optimal viewing.



Figure 11: Camera

3.12 Software

The system would use AI-based hand gesture recognition technology to detect and track the movements of the user's hand. This technology would allow the user to draw and manipulate virtual objects on the computer screen using natural hand movements, without the need for a physical stylus or other input device.

3.13 User Interface

The user interface would consist of a virtual drawing canvas, which would be displayed on the computer screen. The canvas would provide the user with a range of drawing tools and settings, such as brush size, opacity, and color that could be adjusted using natural hand gestures.

3.14 AI Algorithms

The system would incorporate AI algorithms to analyze the user's hand movements and translate them into virtual drawing strokes. These algorithms would be trained on large datasets of hand gestures, allowing them to recognize a wide range of movements and

translate them into accurate and natural-looking drawings.

3.15 Training Data

To train the AI algorithms, the system would require a large dataset of hand gesture data. This data could be collected using a variety of methods, such as motion capture or manual annotation, and would need to be carefully curated and labeled to ensure accuracy.

3.16 Integration with Other Software

To maximize the system's versatility and usability, it would need to be integrated with other software tools and platforms. For example, the system could be integrated with popular digital art software such as Adobe Photoshop or Autodesk Sketchbook, allowing users to create digital art using natural hand gestures.

3.17 Hand Gesture Recognition Technology

The hand gesture recognition technology used in the system would need to be able to accurately detect and track the user's hand movements in real-time. This could be accomplished using a variety of methods, such as computer vision algorithms or machine.

3.18 Natural Hand Gestures

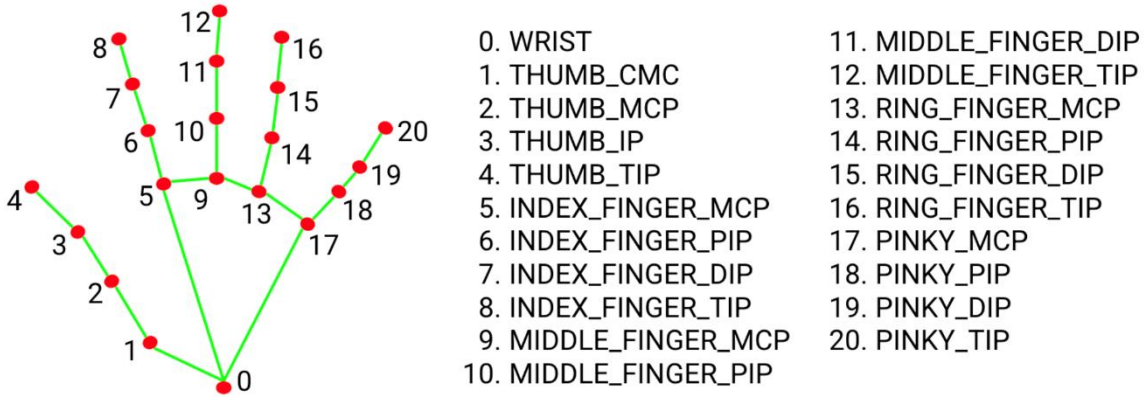


Figure 12: Hand Detection

The system would need to be designed to recognize a range of natural hand gestures, such as finger movements, hand tilts, and wrist rotations. This would allow users to draw and manipulate virtual objects using a variety of intuitive and natural gestures.

3.20 Real-Time Feedback

The system would need to provide real-time feedback to the user, allowing them to see their virtual drawing as they create it. This could be accomplished using a high-speed display or other visual feedback mechanisms.

CHAPTER 4

TESTING

4.1. Black box testing

In black box testing, also known as "Behavioral Testing," the tester does not understand the internals of the product being evaluated. These examinations could or might not be accurate, but in general they are. Here are some methodsthat can be used to make black-box tests:

Equivalence by partitioning input values into valid and invalid groups, and thenselecting test data from each group, partitioning is a technique for building software tests. When developing software tests, Boundary Value Analysis is used to determine the input value bounds and then choose test values that lie on,near, or beyond those bounds.

4.2 Module testing

Module testing is a type of software testing where individual units or components of the software are tested. The purpose of module testing is to isolate a section of code and verify its correctness. Module testing is usually performed by the development team during the early stages of software development

4.2.1 Unit Testing

A unit test is the smallest part of an application that can be tested. This could be a function, a class, a procedure, or an interface. Unit tests are written and run by software developers to make sure that the code works as expected and matches its design and requirements. Each part of this VM prototype is also tested. We broke modules into smaller parts and then tested each part on its own for mistakes, performance, efficiency, and response time.

4.2.2 To accessing camera

```
import cv2
cap=cv2.VideoCapture(0)
while True:
ret,img=cap.read()
cv2.imshow('webcam',img)
k=cv2.waitKey(10)
if k==27:
break; cap.release()
cv2.
```

4.2.3 Color detection code testing

```
imgHSV= cv2.cvtColor(img,cv2.COLOR_BGR2HSV)
create the Mask
mask=cv2.inRange(imgHSV,lowerBound,upperBound)#morphology
maskOpen=cv2.morphologyEx(mask,cv2.MORPH_OPEN,kernelOpen)
maskClose=cv2.morphologyEx(maskOpen,cv2.MORPH_CLOSE,kernelClose)
maskFinal=maskClose
conts,h=cv2.findContours(maskFinal.copy(),cv2.RETR_EXTERNAL,cv2.CHAI
N_APPROX_NONE)
if(len(conts)==2): if(pinchFlag==0):
pinchFlag=1 mouse.release(Button.left)
x1,y1,w1,h1=cv2.boundingRect(conts[0])
x2,y2,w2,h2=cv2.boundingRect(conts[1])
cv2.rectangle(img,(x1,y1),(x1+w1,y1+h1),(255,0,0),2)
cv2.rectangle(img,(x2,y2),(x2+w2,y2+h2),(255,0,0),2) cx1=x1+w1/2
cy1=y1+h1/2 cx2=x2+w2/2 cy2=y2+h2/2
cx=(cx1+cx2)/2 cy=(cy1+cy2)/2
cv2.line(img, (int(cx1), int(cy1)), (int(cx2), int(cy2)), (255, 0, 0), 3,4)
#cv2.line(img, (cx1,cy1), (cx2,cy2),(255,0,0),2)
#cv2.circle(img, (cx,cy),2,(0,0,255),2)
mouseLoc=(sx-(cx*sx/camx), cy*sy/camy) mouse.position=mouseLoc
#mouse.position!=mouseLoc
#while mouse.position!=mouseLoc:
# pass
#cv2.circle(img, (cx,cy),2,(0,0,255),2)
mouseLoc=(sx-(cx*sx/camx), cy*sy/camy) mouse.position=mouseLoc
#mouse.position!=mouseLoc
#while mouse.position!=mouseLoc:
# pass
#cv2.circle(img, (cx,cy),2,(0,0,255),2)
mouseLoc=(sx-(cx*sx/camx), cy*sy/camy) mouse.position=mouseLoc
```

#mouse.position!=mouseLoc

#while mouse.position!=mouseLoc:

4.2.4 Integration Testing

When it comes to software testing, "Integration Testing" is the method through which individual components are combined and evaluated as a whole. The purpose of this kind of testing is to uncover problems with the interplay between different components.

Examples of Integration Test Cases for the Following

- Moving the cursor
- You can drag
- Drop function

Table 4.2.4: Integration Test case

No.	Test case/Testscript	Expected result	Result
1.	Check the movement along with the drag function	Move and drag the icon	Icon dragged successfully
2.	Check the Movement alongwith the drop function	Move and drop the icon	Icon dropped successfully

Table 1: Integration Test Case

4.2.5 System testing

After the system has been built, the next step is to test it. Even if the correct algorithms are used to build the system, it may not be reliable. To make the system acceptable, it is very important to check the results. Before putting the system to work, it needs to be checked to make sure that it is completely within its limits and gives the right results.

Implementation means putting into practice the new system. It could mean putting the whole computer system in place or just putting in one small part of it. One of the most important things to do is to change a manual system into a computerized data processing system.

The implementation phase of our project is the time between when the test design is

approved and when it works well with the help of a user and operation manual.

After the coding competition, review the prototype and make sure that all events, procedures, outputs, inputs, and movements have been checked. We check the detection to make sure it works, the recognition to make sure it's done on time, and the report information for mistakes. Testing starts with each part and moves toward putting the whole computer-based system together

There are different goals and aims for testing. Here are the most important goals of testing: Finding any mistakes that the programmer might make while making the prototype. Getting people to trust and telling them about the level of quality. To prevent defects.

4.2.6 Test Cases

Windows name: Accessing the camera UC No: 01 User Name

Valid: {video capture}invalid: Empty

Table .4.2.6 I: Testing Video Capturing

Test Case ID	Input	Use case Format	Expected Output	Actual Output
01	Video Frame	{mp4}	Capture video	Capture video

Table 2: Testing Video Capturing

UC No: 02 detecting objectsValid: {color detections} Invalid: noise

Table 4.2.6 II: detecting objects

Test Case ID	Input	Use case Format	Expected Output	Actual Output
01	Color	{mp4}	Detecting color	Detecting color

Table 3: Detecting objects

UC No: 03 joints between objects/colorValid: {creating a line}

Invalid: empty

Table 4.2.6 III: joints between object/color

Test Case ID	Input	Use case Format	Expected Output	Actual Output

01	Color	{mp4}	Creating a line between objects	Creating a line between objects

Table 4: Joints

UC No: 04 midpoints Valid: {creating mid-point} Invalid: empty

Table 4.2.6 IV: midpoint of the line

Test Case ID	Input	Use case Format	Expected Output	Actual Output
01	color	{mp4}	Finding a midpoint on line	Creating a midpoint on line

Table 5: midpoint of the line

UC No: 05 cursor movement Valid: {cursor movement along with midpoint}invalid: empty

Table 4.2.6.V: Movement of the Cursor

Test Case ID	Input	Use case Format	Expected Output	Actual Output
01	Color	{mp4}	Cursor movement	Cursor movement

Table 6: movement of the cursor

4.2.7 Automated Testing

Automated checking out is the execution of your test plan (the components of your utility you need to check, the order in which you need to test them, and the anticipated responses) through a script as opposed to a human. Python already comes with a fixed of gear and libraries to help you create computerized checks for your utility.

4.3 Branch Testing

Branch coverage is a testing method, which aims to ensure that each one of the possible branch from each decision point is executed at least once and thereby ensuring that all reachable code is executed.

That is, every branch taken each way, true and false. It helps in validating all the branches

in the code making sure that no branch leads to abnormal behavior of the application.

4.4.1. Use Case Access Camera

Table 4.2: Camera Access scenarios

Use Case Title	Access camera
Use Case Id	1
Description: How the user will access the camera	
Primary Actor:	User
Pre-Conditions: 1. Prototype is working on runtime.	
Post Conditions: 1. Successfully using the prototype.	
Authority: user	
Modification History: 1.0 **Author**: Tufail Ahmad Khan, Sawera Qamar, Muhammad Imran	

Table 7 Access camera

4.3. Use Case Movement

Table 4.3.2: Movement

Use Case Title	Movement
Use Case Id	2
Description: cursor moves along with the movement of the hand.	
Primary Actor	User
Pre-Conditions: 1. User should have a pointed color on the figure.	
Post Conditions: 1. Successfully moving the cursor.	
Authority: User	
Author: Tufail Ahmad Khan, Sawera Qamar, Muhammad Imran	

Table 8 Movement

4.4. Use Case Drag

Table 4.3.3: Drag Scenarios

Use Case Title	Drag
Use Case Id	3
Description: This use case describes the process of dragging the icons.	
Primary Actor	User
Pre-Conditions: 1.. User should have a pointed color on the finger. 2. User should have proper finger positions.	
Post Conditions: 1. Icons are dragged Successfully.	
Authority: User	
Modification History: 1.0 Author: Tufail Ahmad Khan, Sawera Qamar, Muhammad Imran	

Table 9 Drag

Table 4.3.4: Drop scenarios.

Use Case Title	Drop
Use Case Id	4
Description: This use case describes the process of dropping the icons.	
Primary Actor	User
Pre-Conditions: 1.. User should have a pointed color on the finger. 2. User should have proper finger positions.	
Post Conditions: 1. Icons are dropped Successfully.	
Authority: user	
Modification History: 1.0 Author: Tufail Ahmad Khan, Sawera Qamar, Muhammad Imran	

Table 10 Drop

1.4. Tools used

An algorithm for smart recognition

Due to the fact that the current recognition process only works within a 25cm radius, it needs a drag-and-drop function to increase the distance it can cover. This way, it can automatically change the focus rate based on how far away the user is from the webcam.

Better Performance

Several hardware factors contribute to response time, including the capabilities of the camera, the quantity of RAM, and the processing speed of the system. Therefore, the application may perform better on a high-quality computer equipped with a camera that handles varying levels of illumination with ease.

Adding events for clicking

We didn't have enough time or money to add these features to this project. Because information technology is growing quickly, this industry wants more and more, and we are working hard to add clicking events to virtual mice.

- Left-clicking
- Right-clicking

CHAPTER 5

RESULTS OR SCREENSHOTS

5.1 Real-time optical image capture

The frame is recorded in real-time using (cv2.VideoCapture (0); and then each image is flipped and compressed to a reasonable size using vid cod = cv2.VideoWriter force(*'XVID').



Figure 13:Accessing and capturing video

Real-time optical image capture requires consideration of camera hardware capabilities, image resolution, frame rate, and processing system performance.

Efficient algorithms, hardware acceleration, and optimization techniques are utilized to ensure timely and seamless image capture and processing.

These techniques help minimize latency and delays, allowing for immediate access to visual information.

Camera hardware features such as sensor size, pixel count, and lens quality directly impact the quality of captured images.

Striking a balance between image resolution and real-time performance is crucial to maintain smooth operation.

Higher frame rates provide a smoother video stream and reduce motion blur, enhancing the overall visual experience.

The implementation of efficient algorithms optimizes computation time and maximizes processing efficiency.

Table 5.1.1: Mouse Actions

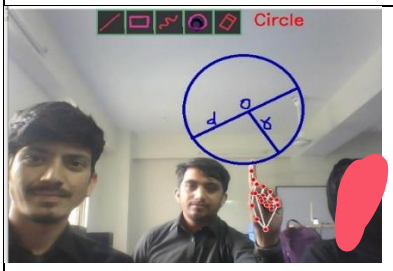
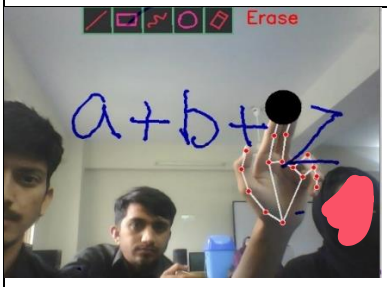
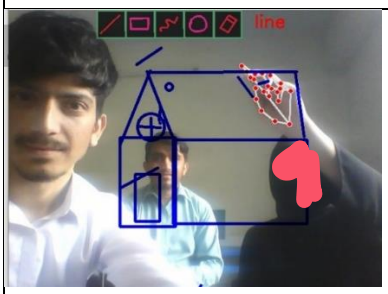
Sample Layout	Tools	Color Combinations
 *Figure 14: Circle*	Circle	First Colors (e.g.Green)
 *Figure 15: Eraser*	Eraser	first Color + Second Color (pinch Gesture)(e.g. Blue + Green)
 *Figure 16: Line*	Line	first Color + Second Color (Un-pinchGesture) (e.g. Blue + Green)

Table 11 Tools Description

5.1.2 User Interface

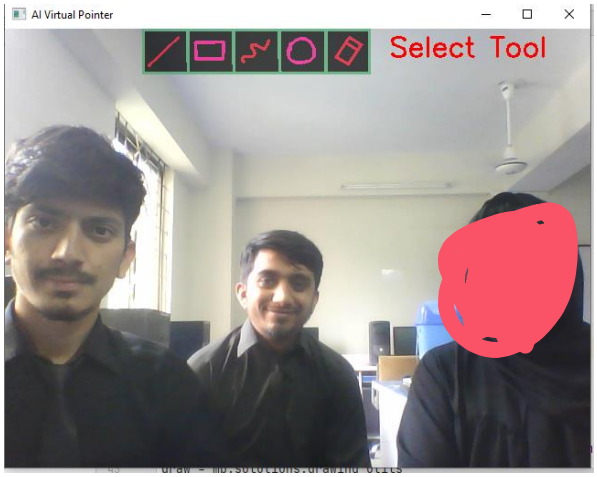


Figure 17: User Interface

CHAPTER 6

CONCLUSION

6.1. Conclusions

It's not shocking that in the field of (HCI), when all mouse motions may be done with the flick of a finger, the physical mouse will eventually be replaced by a virtual non- physical mouse. Our goal was to develop a color-recognition system that could function as a mouse replacement without compromising on precision or speed. The software is sensitive to color combinations and may translate them into action with the mouse. Accuracy and efficiency are crucial to make the program as helpful as a real mouse, thus a few approaches were required. As a first step, we take an average of the coordinates of the colors that determine the cursor's motions. Given that even a little shift in your hand position might cause the cursor to shift in an unexpected direction, it's important to minimize and stabilize the sensitivity of cursor movements.

Aside from that, a variety of color schemes was included, and the distance between any two colors in a scheme was determined. As the separation between two colors determines whether mouse functions are activated, this was done.

This implementation intends to reduce the complexity of the program's operation. That way, true mouse functions may be activated with little guesswork. Calibrations were also included to help make color tracking more precise and adaptable. Because of this, users may assign whatever color they choose to each mouse button, so long as the colors don't share an RGB value (e.g. blue and sky-blue). Additionally, adaptive calibrations were implemented. As a result, the application may store many sets of HSV values from various perspectives for later use in the identification process. In general, technological advancements have boosted societal efficiency and quality of life. It's not the other way around.

As a result, communities shouldn't cling to antiquated technology while resisting the benefits of more modern innovations. They should instead make adjustments to their habits and routines that will allow them to be more effective and efficient.

6.2. Limitations

There are a few issues with this project that can make it difficult to identify colors. One of the issues is the setting in which the recognition stage takes place. The recognition procedure is very light-dependent.

The acquired frames may not have the intended hues if the situation is too light or dark. Besides, distance is another component that might modify the outcomes of color recognition. As the present detecting range only extends out to 25 cm, any color display

beyond that distance will be disregarded as background noise. There is a significant hardware dependency on the program's performance.

The effectiveness of the application may, for instance, depend on the processing speed and/or the resolution of the camera. As a result, when both the processing speed and the resolution are poor, it takes more time to process a single frame. Sometimes colors detection often create a minor problem because of same color in background.

Webcam detects those colors which is problem for accuracy so if we are using this we need colorless background. But this is not a big issue we can remove rgb color detection API and used hand gesture process in which the cam detects full hand and all fingers.

6.3. Future Work

There are a few things that need to be added or changed to make the application easier to use, more accurate, and more adaptable to different situations. Here are the changes that need to be made and the features that are needed:

CHAPTER 7

REFERENCE

References

1. Kenneth E Kendall & Julie - System Analysis and Design (4th edition).
2. Kumar N, M. (2011). Manual Testing: Agile software development. [online] Manojforqa.blogspot.com. Available at: http://manojforqa.blogspot.com/2011/09/agile-software-development.html [Accessed 27 Aug. 2015].
3. Activity Diagram https://app.diagrams.net/
4. Sequence diagram- https://app.diagrams.net/
5. Black box and white box testing - https://www.youtube.com/watch?v=49i26aesGaU
6. https://www.youtube.com/channel/UCm9daSbiQK6UoOlXxmr_rnQ
7. Park, H. (2008). A method for controlling mouse movement using a real-time camera. Brown University, Providence, RI, USA, Department of computer science.
8. https://www.youtube.com/watch?v=8gPONnGIPgw